EVALUATION

Four Decision Models, One Protocol

A paired evaluation of CLEF-Flash, Jev, Laya, and CLM on intent triage, reranking, knowledge, routing, and tool selection

Isham Rashik · AI Engineer
Engineering AI with Clarity
NLP • Computer Vision • Fine-Tuning • RAG • Agentic AI

Version: v1.0 · October 12, 2026

github.com/di37 · huggingface.co/disham993
linkedin.com/in/isham-rashik-5a547711b
Dubai, UAE

Abstract

Decision models answer typed questions, such as "which of these options applies?", with a probability for every option instead of free text. That makes them natural routers, triage classifiers, and tool selectors inside agentic systems. This report compares four of them, CLEF-Flash, Jev, Laya, and CLM, by sending each one identical requests through the same SystemOne decision API and the same sequential client. Five suites cover Banking77 intent triage (154 messages, 77 options), Amazon ESCI reranking (50 queries, 10 products each), MMLU (200 questions from ten subjects), a curated 90-prompt model-routing set, and the full 200-case BFCL v3 multiple tool-selection split. All 2,776 measured requests completed without errors or retries. CLEF-Flash and Jev form a clear top tier. CLEF-Flash reaches 93.5% on Banking77 against Jev's 80.5% (exact McNemar test, Holm-adjusted $p = 1.1 \times 10^{-5}$), while Jev leads on the other four suites by margins that are not significant (Holm p between 0.25 and 1.0). Both top models significantly outperform Laya and CLM on every suite. Laya is the fastest model (8–20 ms median latency) and reaches 94.0% on tool selection, but trails far behind on MMLU and routing. The zero-shot CLM checkpoint collapses onto two intents on Banking77 and lands within 0.008 nDCG of a random ordering on ESCI. Alongside the headline numbers, the report gives Wilson intervals, paired bootstrap tests, calibration metrics, and an error analysis showing that two of the five routing errors made by the top models coincide with an alternative reference label.

Contents

1 Introduction **4**

2 Related Work **5**

3 Methodology **5**

3.1 The SystemOne decision protocol 6

3.2 Task formulation 7

3.3 Fairness controls 8

3.4 Statistical analysis 9

4 Experimental Setup **10**

4.1 Datasets and sampling 10

4.2 Providers and serving configuration 11

4.3 Execution record 12

5 Results **13**

5.1 Headline comparison 13

5.2 Pairwise significance 13

5.3 Intent triage and knowledge . . . 15
5.4 Reranking . . . 16
5.5 Model routing . . . 17
5.6 Tool selection . . . 18
5.7 Calibration . . . 19
5.8 Latency . . . 20

6 Discussion **21**
6.1 Where the tiers separate . . . 21
6.2 Threats to validity . . . 21
6.3 Practical guidance . . . 22
6.4 Future work . . . 22

7 Conclusion **23**

A All pairwise McNemar tests **23**

B MMLU accuracy by subject **24**

C Reproducing the analysis **25**

D References **25**

1 Introduction

Agentic systems make many small decisions before they generate any text: which queue a support ticket belongs to, which tool a request needs, and how much model capacity a prompt deserves. A decision model answers those questions directly. It receives a *state* and a set of named *questions*, each listing its options, and returns a choice together with a probability for every option. The four models studied here, CLEF-Flash [1], Jev [2], Laya [3], and CLM [4], all expose this interface through the same SystemOne endpoint. A shared protocol makes a like-for-like comparison possible at the level of requests rather than prompts.

Comparisons between such models are easy to get wrong. Option order, question wording, client timeouts, warm server caches, and GPU sharing can each shift accuracy by several points, and a single unpaired accuracy figure hides how often two models disagree on the same case. This study holds every one of those factors fixed and measures what remains, using paired tests that compare the models case by case.

The report makes four contributions:

1. A controlled comparison in which every provider receives the same state, questions, option text, and option order from one shared client, with the three local models served one at a time on the same GPU (Section 3).
2. Five suites totalling 694 measured requests per provider, with every per-case output and run manifest saved, so each number in this report can be recomputed without calling a model (Section 4).
3. Paired significance tests with family-wise correction, interval estimates, calibration metrics, and explicit chance baselines, including a random-permutation floor for ESCI and a uniform-random baseline for BFCL (Section 5).
4. An error analysis that separates genuine routing failures from label ambiguity in the curated routing set (Section 5.5).

Section 2 positions the work, Section 3 defines the protocol, task formulations, and statistics, Section 4 describes the data and serving setup, Section 5 reports the findings, and Section 6 discusses limitations before Section 7 concludes. The next section starts with the work this evaluation builds on.

2 Related Work

Three lines of work shape this evaluation: model routing, the public benchmarks reused here, and the statistics of comparing classifiers. This section covers each in turn.

Routing and cascades. FrugalGPT [5] cascades calls across language models of increasing cost and stops once an answer is judged reliable. RouteLLM [6] learns routers between a strong and a weak model from preference data. The routing suite in this report asks a narrower question: given an explicit rubric, can a decision model name the smallest capability tier that can complete a request?

Benchmarks. Banking77 [7] is a fine-grained intent detection set with 77 banking intents. The Shopping Queries Dataset [8] labels query and product pairs as Exact, Substitute, Complement, or Irrelevant (ESCI). MMLU [9] measures knowledge across 57 subjects with four-way multiple choice. The Berkeley Function Calling Leaderboard [10], from the Gorilla project [11], evaluates tool calls; its *multiple* split offers several candidate functions per request. Every suite here is a reduced or reframed version of its source (Section 3.2), so the scores are diagnostic and not comparable to leaderboard numbers.

Evaluation methodology. Dietterich [12] recommends McNemar's test [13] for comparing two classifiers on the same test set, and Holm's procedure [14] controls the family-wise error rate across many pairwise tests. Wilson intervals [15] remain accurate near 0% and 100%, where the normal approximation fails. Ranking quality uses nDCG [16] with the exponential gain popularised by RankNet [17], compared through a paired bootstrap [18]. Calibration uses log loss, the Brier score [19], and expected calibration error [20, 21]. Finally, language models are known to be sensitive to the order in which options appear [22, 23], which motivates the deterministic option shuffle described next.

With that context in place, the next section defines exactly what each model was asked and how the answers were scored.

3 Methodology

The methodology has four parts: the request protocol shared by all models, the way each benchmark is cast into that protocol, the controls that keep the comparison fair, and the statistics used to compare models. Figure 1 shows how the parts connect.

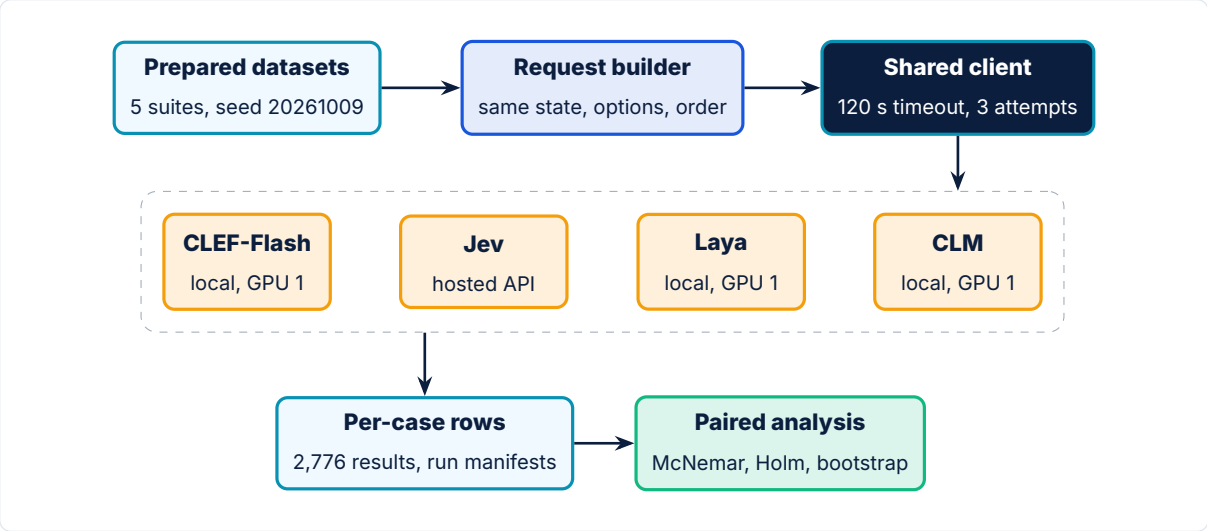


Figure 1: Evaluation harness. One request builder and one client serve all four providers; the three local models run one at a time on the same GPU, and every answer is stored as a per-case row before any statistic is computed.

3.1 The SystemOne decision protocol

Every provider accepts the same request shape at `POST /v1/systemone`. The `state` field holds the input as a JSON object. The `questions` field maps question names to question definitions; this study uses only the `choice` type, whose `criteria` map each option name to a short description. Code 1 shows an MMLU request in this form.

```json
{
  "state": {
    "subject": "world religions",
    "question": "What is the most important festival for Sikhs?"
  },
  "questions": {
    "answer": {
      "type": "choice",
      "instructions": "Select the one correct answer to the multiple-choice question.",
      "criteria": {"A": "Holi", "B": "Divali", "C": "Baisakhi Day", "D": "Hola Mahalla"}
    }
  },
  "model": "clef-flash"
}
```

Code 1: An MMLU request in SystemOne form. Options are shown in A–D order; the harness shuffles them per case before sending

The response carries one answer per question, with the chosen option and a probability for each option, plus the serving model's identifier and token usage. Code 2 reproduces a real response from the routing run.

```json
{
  "model": "jev-1.13.0",
  "answers": {
    "route": {
      "type": "choice", "choice": "small", "confidence": 1.0,
      "probabilities": {"small": 1.0, "medium": 0.0, "large": 0.0}
    }
  },
  "usage": {"input_tokens": 524, "output_tokens": 38}
}
```

Code 2: A saved Jev response to routing case small-001, a title-case conversion request

Because the probabilities are part of the protocol, every suite can be scored for both accuracy and calibration. The next subsection explains how each benchmark is mapped onto this request shape.

3.2 Task formulation

Each suite becomes one SystemOne request per case, summarised in Table 1. The formulations are deliberately plain: option names and descriptions come straight from the source data, so no provider benefits from prompt engineering.

Table 1: The five evaluation suites

Suite	Source	Cases	Options	Primary metric
Banking77 triage	[7]	154	77	Accuracy
ESCI reranking	[8]	50 queries	4 per product	nDCG@10
MMLU	[9]	200	4	Accuracy
Model routing	curated	90	3	Accuracy
BFCL v3 multiple	[10]	200	2–4	Accuracy

Banking77 triage. The state is the customer message. A single `queue` question offers all 77 intents, each described by its label with underscores replaced by spaces (for example, “card arrival”). Secondary metric: macro-F1.

MMLU. The state holds the subject and the question; the four answer texts are the options A–D.

ESCI reranking. One request per query asks a separate four-way relevance question for each of its ten products. Each product j receives an expected-relevance score from the returned probabilities p_j:

$$s_j = \sum_{g \in \mathcal{G}} r(g)\, p_j(g), \tag{1}$$

where $\mathcal{G}$ is the set of ESCI grades and r maps Exact, Substitute, Complement, and Irrelevant to 3, 2, 1, and 0. Products are ranked by descending s_j, and the ranking is scored with

$$\text{nDCG@10} = \frac{\text{DCG}}{\text{IDCG}}, \qquad \text{DCG} = \sum_{i=1}^{10} \frac{2^{r_{(i)}} - 1}{\log_2(i+1)}, \tag{2}$$

where $r_{(i)}$ is the gold grade of the product at rank i and IDCG is the DCG of the ideal order. The exponential gain gives Exact, Substitute, Complement, and Irrelevant gains of 7, 3, 1, and 0. The secondary metric is the mean reciprocal rank (MRR) of the first Exact product.

Model routing. The state is the user prompt. A single `route` question asks for the *minimum* capability tier (`small`, `medium`, or `large`) that can reliably complete the whole request, with four tie-breaking rules: prefer the lowest sufficient tier; high consequence raises a request to `large`; several otherwise-medium deliverables together are `large`; and input length alone does not raise the tier when the work is mechanical.

BFCL tool selection. The state is the flattened conversation. The options are the exact candidate function names, each described by its documentation string and its JSON parameter schema. Only the selected function name is scored, not its arguments.

These formulations fix what is asked. The next subsection fixes how it is asked, so that the only remaining difference between runs is the model.

3.3 Fairness controls

Five controls keep the comparison fair.

- **Identical requests.** Every provider receives the same state, questions, option text, and option order. The only provider-specific fields are each vendor's model identifier and, for Laya, its documented token-budget fields (Section 4.2).
- **Deterministic option order.** For Banking77, MMLU, ESCI, and BFCL the option order is shuffled per case, with a seed derived from the case identifier c:

  $$\pi_c = \text{Shuffle}\big(\mathcal{O}_c;\ \text{seed} = \text{SHA-256}(\texttt{20261009:}\, c)\big). \tag{3}$$

  The order is therefore random with respect to the gold label but identical across providers. The routing question keeps a fixed order, discussed in Section 6.2.
- **One client.** All requests go through one sequential HTTP client with a 120 s timeout and up to three attempts with exponential backoff (0.5 s, then 1 s). Latency is measured on the client for the successful attempt.
- **Cold-cache protection.** Each provider run starts with one warm-up request that is not part of any dataset, so no server-side cache can answer a measured case.

- **One GPU, one model at a time.** CLEF-Flash, Laya, and CLM ran sequentially on the same NVIDIA RTX 4090, each from a freshly restarted server. Jev ran remotely in the same session.

Each provider run followed the same procedure:

1. **Start the server fresh**: Restart the provider's container on the benchmark GPU so no state carries over.
2. **Record the environment**: Append the endpoint, model identifier, extra request fields, retry policy, health response, and GPU memory to the run manifest.
3. **Warm up**: Send one request that belongs to no dataset.
4. **Run the suites**: Send every pending case sequentially and append each answer as a row the moment it arrives.
5. **Stop the server**: Shut the container down before the next local model starts.

These controls make the per-case outputs directly comparable. The final methodological question is how to decide whether an observed difference is real.

3.4 Statistical analysis

All comparisons are paired: the same cases go to every model, so the tests compare models case by case rather than through independent samples.

Interval estimates. Accuracy on Banking77 and MMLU is reported with a 95% Wilson interval,

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad z = 1.96, \tag{4}$$

and tested against chance with a one-sided exact binomial test (chance is 1/77 for Banking77 and 1/4 for MMLU).

Paired accuracy tests. For two models on the same n cases, let b count cases only the first model gets right and c cases only the second gets right. The exact two-sided McNemar test is

$$p = \min\left\{1,\ 2\sum_{k=0}^{\min(b,c)} \binom{b+c}{k} 2^{-(b+c)}\right\}. \tag{5}$$

Paired ranking tests. For ESCI, the per-query nDCG difference between two models is resampled over queries $B = 20{,}000$ times with replacement. The 2.5th and 97.5th percentiles of the resampled mean differences Δ_b^* give the confidence interval, and the two-sided p-value is

$$p_{\text{boot}} = \min\left\{1,\ 2\min\left(\frac{1}{B}\sum_{b=1}^{B}\mathbf{1}[\Delta_b^* \le 0],\ \frac{1}{B}\sum_{b=1}^{B}\mathbf{1}[\Delta_b^* \ge 0]\right)\right\}. \tag{6}$$

Multiple comparisons. Each suite has $m = 6$ provider pairs. With the raw p-values sorted as $p_{(1)} \le \cdots \le p_{(m)}$, Holm's adjusted values are

$$\tilde{p}_{(i)} = \max_{j \le i} \min\{1,\ (m - j + 1)\, p_{(j)}\}. \tag{7}$$

A difference is called significant when $\tilde{p} < 0.05$.

Calibration. For N cases with gold labels y_i, options $\mathcal{K}_i$, and returned probabilities p_i,

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^{N} \log \max\big(p_i(y_i),\ 10^{-12}\big), \tag{8}$$

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^{N} \sum_{k \in \mathcal{K}_i} \big(p_i(k) - \mathbf{1}[k = y_i]\big)^2, \tag{9}$$

$$\text{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{N} \big|\text{acc}(B_b) - \text{conf}(B_b)\big|, \tag{10}$$

where ECE groups cases into ten equal-width bins B_b by their top-class probability. ECE is computed for routing and tool selection; log loss and Brier are computed for every single-choice suite. Every provider returned a complete probability distribution on every case, so no case drops out of these metrics.

With the protocol and statistics defined, the next section describes the data and the serving setup that produced the results.

4 Experimental Setup

This section covers the prepared datasets, the four serving configurations, and the execution record of the run.

4.1 Datasets and sampling

The three public subsets were sampled once with seed 20261009 and saved, together with SHA-256 hashes of the source files (Table 2). BFCL uses the complete official split. The routing set was written for this study.

Table 2: Data sources and sampling. Hashes are the first eight hex digits of each source file's SHA-256

Suite	Source files	Sampling	SHA-256
Banking77	`mteb/banking77` test [24]	2 messages per intent, 77 intents	9575f636
ESCI	`tasksource/esci` test, shard 0 [25]	50 of 1,855 eligible US queries; 10 products each, at least one Exact and one Irrelevant	01b0da5f
MMLU	`cais/mmlu` all, test [26]	20 questions from each of 10 fixed subjects	74a41822
BFCL	v3 multiple inputs and answers [10]	all 200 cases	aef16815, 244e00ce
Routing	curated	30 prompts per tier, 87 task categories	n/a

Two properties of the samples matter for interpretation. First, 218 of the 500 sampled ESCI products are labelled Exact, which raises the score of a random ordering (Section 5.4). Second, the routing file carries an alternative reference label for every prompt, which disagrees with the gold label on 5 of the 90 prompts; Section 5.5 uses it to examine routing errors. BFCL cases offer two candidates in 79 cases, three in 85, and four in 36, so choosing uniformly at random would score $(79/2 + 85/3 + 36/4)/200 = 38.4\%$.

4.2 Providers and serving configuration

Each local model ran through its vendor's own serving code with the configuration its documentation recommends (Table 3).

Table 3: Serving configuration. The model column shows the identifier each server reported in its responses

Provider	Serving stack	Model	Extra fields
CLEF-Flash	Qwen3.5-9B backbone with a joint schema head, loaded by the model's own release code; transformers 5.10.2, PyTorch 2.11	`clef-flash`	none
Jev	TypeSafe hosted API	`jev-1.13.0`	none
Laya	`laya.serve` 0.3.21 with its Router choosing among English, multilingual, and typed-decision checkpoints	`laya-rl-agent`	token budget
CLM	`clm-serve` 0.1.0 heads over Qwen3-8B [27] last-token embeddings served by vLLM 0.19.0 [28] in bfloat16	`clm-latest`	none

Laya's request omits a model identifier, so its Router selects the checkpoint; it chose the English checkpoint for all of these English inputs. Laya also received `max_len` = 2048 and `head_max_len` = 1024. At Laya's suggested `head_max_len` of 512, Banking77's 77 options collapse to 72 distinct spans; 1024 is the smallest budget tested at which Laya reports no merged options. CLM ran its projection heads on the CPU with its action cache set to zero, behind a vLLM encoder limited to 2,048 tokens.

4.3 Execution record

The run manifests record the whole evaluation on 11 October 2026 between 20:24 and 20:33 UTC, in the order Jev, CLM, CLEF-Flash, Laya. Each provider answered 404 public-suite cases, 90 routing prompts, and 200 BFCL cases: 694 measured requests per provider and 2,776 in total. Every request succeeded on its first attempt. The GPU snapshots taken at the start of each run show 18,675–19,647 MiB in use on the benchmark GPU for CLEF-Flash, 21,579–21,679 MiB for CLM, and 3,375–4,359 MiB for Laya. A second RTX 4090 in the same host held 20,636 MiB for an unrelated workload throughout.

Complete, error-free run

All 2,776 measured requests returned a valid answer on the first attempt, and every answer included a probability for every option. No case is missing from any metric in this report.

With a complete and clean set of per-case answers, the next section turns to what they show.

5 Results

The results lead with the headline comparison and its significance, then examine each suite, and close with calibration and latency.

BANKING77, CLEF-FLASH
93.5% +13.0 vs Jev

BFCL, JEV
100% 200/200

MEDIAN LATENCY, LAYA
8.4 ms

Two tiers emerge. CLEF-Flash and Jev are statistically indistinguishable on four of five suites; the one significant gap between them is fine-grained intent triage, where CLEF-Flash leads by 13 points. Both are significantly better than Laya and CLM on every suite.

5.1 Headline comparison

Table 4 lists the primary metric for every suite, and Figure 2 plots it against each suite's chance or random baseline.

Table 4: Primary metric by suite. Accuracies in percent; best value in bold

Suite (metric)	CLEF-Flash	Jev	Laya	CLM	Chance
Banking77 (accuracy)	**93.5**	80.5	45.5	1.3	1.3
ESCI (nDCG@10)	0.9139	**0.9183**	0.8553	0.8012	0.7940
MMLU (accuracy)	92.5	**93.5**	34.5	29.5	25.0
Routing (accuracy)	96.7	**97.8**	47.8	35.6	33.3
BFCL (accuracy)	98.5	**100.0**	94.0	65.5	38.4

The picture is consistent across suites: the top two models sit near the ceiling everywhere except Banking77, Laya sits between the tiers, and CLM stays close to its baseline on three suites. Whether those visual gaps are real is the subject of the next subsection.

5.2 Pairwise significance

Each suite has six provider pairs, tested with Equation (5) (or Equation (6) for ESCI) and Holm-adjusted within the suite (Equation (7)). Table 5 isolates the top-tier comparison.

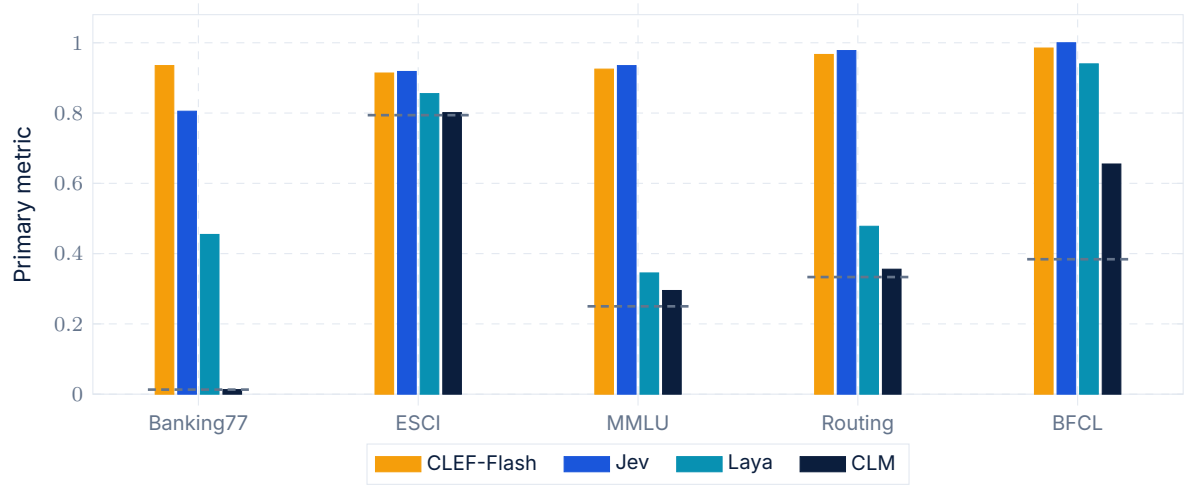


Figure 2: Primary metric by suite. Dashed lines mark chance accuracy (Banking77, MMLU, routing), the uniform-random baseline for BFCL, and the random-permutation nDCG@10 for ESCI.

Table 5: CLEF-Flash vs Jev. "Only" columns count cases one model answers correctly and the other does not; the effect is CLEF-Flash minus Jev

Suite	CLEF only	Jev only	Effect	p	p_{Holm}
Banking77	21	1	+13.0 pts	1.1×10^{-5}	1.1×10^{-5}
ESCI	–	–	-0.0044 nDCG $[-0.042, 0.028]$	0.840	0.840
MMLU	9	11	-1.0 pts	0.824	0.824
Routing	2	3	-1.1 pts	1.000	1.000
BFCL	0	3	-1.5 pts	0.250	0.250

Banking77 is decisive: of the 22 cases on which the two models disagree in correctness, CLEF-Flash wins 21. Everywhere else the discordant counts are small and balanced, and the ESCI interval comfortably spans zero. A larger sample could still reveal a difference on BFCL, where Jev wins all three discordant cases, but three cases cannot establish one.

Table 6: Laya vs CLM. The effect is Laya minus CLM

Suite	Laya only	CLM only	Effect	p	p_{Holm}
Banking77	70	2	+44.2 pts	1.1×10^{-18}	3.3×10^{-18}
ESCI	–	–	$+0.0541$ nDCG $[0.008, 0.104]$	0.019	0.037
MMLU	46	36	+5.0 pts	0.320	0.641
Routing	25	14	+12.2 pts	0.108	0.216
BFCL	64	7	+28.5 pts	1.3×10^{-12}	5.0×10^{-12}

Table 6 shows that Laya is significantly better than CLM on Banking77, ESCI, and BFCL, while MMLU and routing do not separate them. All 20 cross-tier comparisons (CLEF-Flash or Jev

against Laya or CLM, on each of the five suites) are significant after correction; the weakest is CLEF-Flash against Laya on BFCL (11 against 2 discordant cases, Holm $p = 0.045$). Appendix A lists all 24 McNemar tests, and Table 9 gives the six ESCI comparisons.

Reading the pairwise tables

McNemar's test uses only the discordant cases. Two models with similar accuracy can still differ significantly if one wins most disagreements, and a large accuracy gap built on few disagreements can fail to reach significance. The "only" columns show both counts so the reader can judge the evidence directly.

The pairwise tests settle the ranking. The remaining subsections explain where in each suite the models succeed and fail.

5.3 Intent triage and knowledge

Table 7 adds 95% Wilson intervals and chance tests to the Banking77 and MMLU accuracies.

Table 7: Accuracy with 95% Wilson intervals and one-sided binomial tests against chance

Provider	Banking77 [95% CI]	p vs 1/77	MMLU [95% CI]	p vs 1/4
CLEF-Flash	93.5 [88.5, 96.4]	3.0×10^{-257}	92.5 [88.0, 95.4]	8.4×10^{-92}
Jev	80.5 [73.6, 86.0]	6.2×10^{-203}	93.5 [89.2, 96.2]	5.6×10^{-95}
Laya	45.5 [37.8, 53.3]	2.3×10^{-88}	34.5 [28.3, 41.3]	0.0017
CLM	1.3 [0.4, 4.6]	0.596	29.5 [23.6, 36.2]	0.084

On Banking77, the CLEF-Flash and Jev intervals do not overlap, which agrees with the McNemar result. Macro-F1 tells the same story (0.932 for CLEF-Flash, 0.791 for Jev, 0.438 for Laya, 0.001 for CLM). CLM is the striking case: it predicted only two of the 77 intents, `exchange_via_app` for 108 messages and `card_acceptance` for the other 46, and its accuracy is indistinguishable from chance ($p = 0.596$).

On MMLU, the top two models are within one point of each other and both clear 88% at the lower interval bound. Laya is above chance ($p = 0.0017$) but more than 50 points behind; CLM does not reach significance against chance ($p = 0.084$). The subject breakdown in Appendix B shows CLEF-Flash's weakest subject is anatomy (75%), while Jev's is business ethics (80%); with 20 questions per subject, these differences are within noise.

Both suites reward knowledge of the label space. The reranking suite asks a different question: how well the probabilities order products.

5.4 Reranking

Table 8 reports the ranking metrics alongside per-product label accuracy, and Table 9 gives the paired bootstrap comparisons.

Table 8: ESCI ranking and per-product label metrics. The random row is the mean over 20,000 random orderings

Provider	nDCG@10	MRR (Exact)	Label accuracy	Label macro-F1
CLEF-Flash	0.9139	0.8545	**0.568**	**0.532**
Jev	**0.9183**	**0.8670**	0.546	0.489
Laya	0.8553	0.7730	0.428	0.304
CLM	0.8012	0.6329	0.282	0.189
Random ordering	0.7940	0.6277	–	–

Table 9: Paired bootstrap on per-query nDCG@10 (20,000 resamples). A p of $< 10^{-4}$ means no resample crossed zero

Pair (A minus B)	Mean difference	95% CI	p	p_{Holm}
CLEF-Flash minus Jev	-0.0044	$[-0.0422, 0.0282]$	0.840	0.840
CLEF-Flash minus Laya	$+0.0586$	$[0.0210, 0.0965]$	0.0015	0.0045
CLEF-Flash minus CLM	$+0.1127$	$[0.0754, 0.1520]$	$< 10^{-4}$	$< 10^{-4}$
Jev minus Laya	$+0.0630$	$[0.0278, 0.0988]$	0.0006	0.0024
Jev minus CLM	$+0.1171$	$[0.0721, 0.1664]$	$< 10^{-4}$	$< 10^{-4}$
Laya minus CLM	$+0.0541$	$[0.0082, 0.1036]$	0.0185	0.037

The two top models rank products almost equally well, yet they label them differently: CLEF-Flash assigns the exact ESCI grade more often (56.8% against 54.6%), while Jev's probabilities produce a marginally better ordering. Label accuracy stays below 57% for every model, which reflects how ambiguous the Substitute and Complement grades are, even though the rankings are good.

> ⚠ **Read nDCG against its floor**
>
> A random ordering already scores nDCG@10 = 0.794 and MRR = 0.628 here, because 44% of the sampled products are Exact and many queries have several. The usable range is therefore roughly 0.79 to 1.0, not 0 to 1. CLM's 0.801 sits within 0.008 of random, and the 0.06 gap between Laya and the top tier is larger than it looks.

Reranking compares models on how they spread probability. Model routing instead asks them to judge the difficulty of a request, which turns out to separate the tiers sharply.

5.5 Model routing

Table 10 and Figure 3 break routing accuracy down by expected tier.

Table 10: Routing accuracy, macro-F1, per-tier recall, and the distribution of predicted tiers (small-/medium/large; 30 of each is ideal)

Provider	Accuracy	Macro-F1	Small	Medium	Large	Predicted
CLEF-Flash	96.7	0.967	1.000	0.900	1.000	30/27/33
Jev	**97.8**	**0.978**	1.000	0.933	1.000	32/28/30
Laya	47.8	0.438	0.567	0.767	0.100	23/60/7
CLM	35.6	0.249	0.900	0.167	0.000	68/21/1

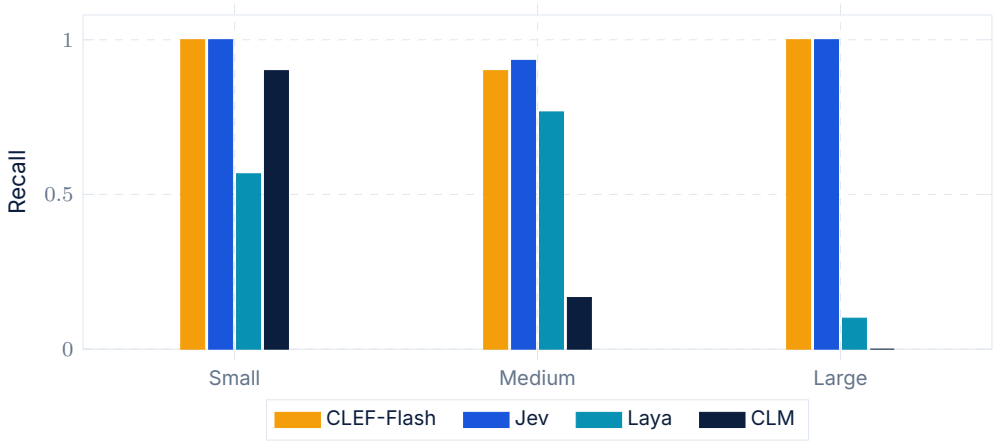


Figure 3: Routing recall by expected tier. Laya funnels most prompts to *medium* and CLM to *small*, so neither recognises large requests.

The weaker models fail in opposite directions. Laya sends 60 of 90 prompts to *medium* and recognises only 3 of 30 large requests; CLM sends 68 prompts to *small* and recognises none. Both patterns would be costly in production: Laya would under-provision difficult work, and CLM would under-provision almost everything.

The top models make only five errors between them, all on medium prompts (Table 11). CLEF-Flash errs upward three times, routing medium work to *large*; Jev errs downward twice, routing it to *small*.

Table 11: Every routing error made by CLEF-Flash or Jev, with the alternative reference label

Case	Task	Gold	Alternative	Model	Predicted
medium-011	Migration notice with impact and rollback contact	medium	medium	Jev	small
medium-019	Secure password-reset flow with token expiry	medium	large	CLEF-Flash	large
medium-020	Regex for ticket IDs, with explanation	medium	small	Jev	small
medium-021	Idempotent, resumable nightly CSV import	medium	medium	CLEF-Flash	large
medium-028	Minor framework upgrade with rollback plan	medium	medium	CLEF-Flash	large

Errors or label ambiguity?

Two of the five errors, medium-019 and medium-020, are prompts where the alternative reference label agrees with the model rather than the gold label. The password-reset prompt is security-sensitive, which the rubric's high-consequence rule could reasonably push to *large*; the regex prompt is close to a single mechanical step. Switching to the alternative labels would not favour either model, though: they rescue one error per model but contradict four answers each model currently gets right, so CLEF-Flash would score 93.3% and Jev 94.4%. The gold labels are curated judgments, and differences of one or two cases between the top models should be read in that light.

Routing judgements depend on understanding what a request entails. Tool selection depends mostly on matching a request to a function description, and the gap between the tiers narrows accordingly.

5.6 Tool selection

Table 12 reports BFCL accuracy and calibration.

Table 12: BFCL v3 multiple tool selection (200 cases)

Provider	Accuracy	Macro-F1	Log loss	Brier	ECE
CLEF-Flash	98.5	0.984	0.0500	0.0129	0.0449
Jev	**100.0**	**1.000**	**0.0027**	**0.0006**	**0.0025**
Laya	94.0	0.936	0.2289	0.1132	0.1138
CLM	65.5	0.635	0.9995	0.5263	0.1578
Uniform random	38.4	–	–	–	–

Jev selects the right function in all 200 cases with almost no probability on the alternatives. CLEF-Flash's three errors are all near-misses between closely related functions: `store_price` against `on_sale` in a video-games module, Biot–Savart against Ampère's law in an electromagnetism module, and a wildlife population-growth function against a broader ecological impact analysis. Laya's 94.0% is its best result in the study and the suite where it comes closest to the top tier. CLM is far above the uniform baseline (65.5% against 38.4%), which shows its encoder carries useful signal even where its zero-shot heads fail elsewhere.

Accuracy alone does not say whether a model's probabilities can be trusted as confidence scores. The next subsection examines that directly.

5.7 Calibration

Table 13 collects log loss and Brier scores for the single-choice suites, and Figure 4 compares each model's mean top-class probability with its accuracy on routing and BFCL.

Table 13: Calibration by suite (lower is better). ECE is reported for routing and BFCL only

Suite and metric	CLEF-Flash	Jev	Laya	CLM
Banking77 log loss	**0.274**	1.998	8.766	5.880
Banking77 Brier	**0.094**	0.291	0.954	1.123
MMLU log loss	**0.244**	0.456	1.393	2.575
MMLU Brier	**0.111**	0.112	0.758	1.050
Routing log loss	0.226	**0.067**	0.909	1.707
Routing Brier	0.093	**0.038**	0.550	0.888
Routing ECE	0.140	**0.044**	0.149	0.358
BFCL log loss	0.050	**0.003**	0.229	0.999

Three patterns stand out. CLEF-Flash has the best log loss and Brier score on Banking77 and MMLU, but it is *underconfident* on routing: its mean confidence is 0.83 while its accuracy is 0.97, which explains its routing ECE of 0.140. Jev shows the opposite failure on Banking77: its log loss of 2.0 comes almost entirely from its 30 errors, where the gold intent receives a mean negative log probability of 9.8 (a probability of roughly 6×10^{-5}), against 0.12 on its correct answers. CLM is *overconfident* on both routing (0.71 mean confidence at 36% accuracy) and BFCL (0.79 at 66%). Laya's Banking77 log loss of 8.8 is consistent with its checkpoint's own warning that the calibration temperature for questions with eleven or more options is invalid and was clamped.

Calibration describes how much to trust a single answer. Latency describes what each answer costs, and the last subsection reports it.

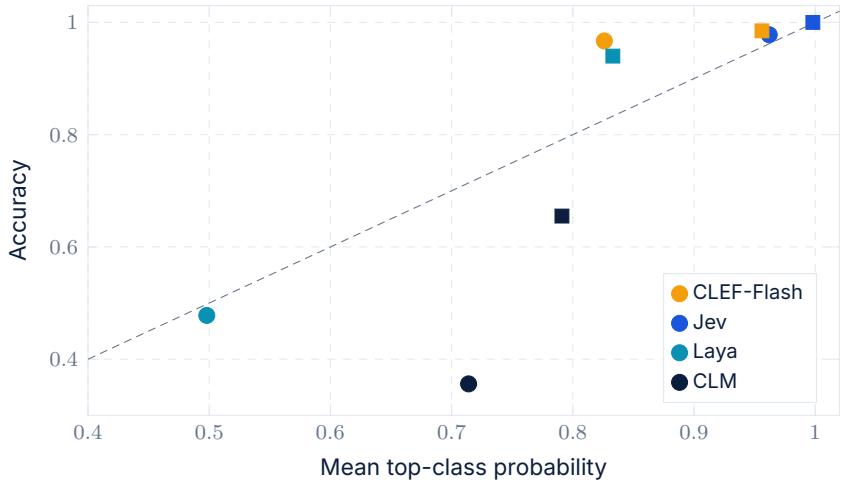


Figure 4: Mean top-class probability against accuracy, computed from the saved per-case rows. Circles are routing, squares are BFCL. Points above the dashed diagonal are underconfident; points below it are overconfident.

5.8 Latency

Table 14 gives client-side median and 95th-percentile latency for every suite.

Table 14: Client-side latency in milliseconds, median / 95th percentile. Jev includes the network round trip to its hosted API

Suite	CLEF-Flash	Jev	Laya	CLM
Banking77	322.6 / 323.7	327.2 / 411.1	16.3 / 17.1	25.7 / 26.6
ESCI	309.0 / 332.1	312.3 / 410.2	19.5 / 21.0	66.8 / 76.2
MMLU	69.1 / 87.5	315.5 / 410.6	8.4 / 9.6	61.3 / 68.9
Routing	86.2 / 86.6	309.4 / 400.2	9.0 / 9.4	28.2 / 30.2
BFCL	102.5 / 140.4	312.0 / 408.4	8.9 / 9.6	68.7 / 74.8

Laya is an order of magnitude faster than every other model on every suite. CLEF-Flash's latency tracks request size: about 70–100 ms for the four-option and tool questions, but over 300 ms for Banking77's 77 options and ESCI's ten questions per request. Jev's latency is nearly constant at about 310 ms, which suggests that network time dominates; it should not be read as a measure of Jev's compute cost.

Results in brief

CLEF-Flash and Jev are tied on knowledge, ranking, routing, and tool selection, and CLEF-Flash is clearly better at fine-grained intent triage. Laya is fast and strong at tool selection but weak at judging difficulty. The zero-shot CLM checkpoint is near chance on three suites. Calibration and latency add trade-offs that accuracy alone hides.

These findings raise the question of why the tiers separate where they do, and how far the

conclusions generalise. The discussion takes up both.

6 Discussion

This section interprets the tier structure, sets out the threats to validity, turns the findings into practical guidance, and lists the open questions.

6.1 Where the tiers separate

The suites differ in how much they demand beyond matching text to an option. BFCL tool selection is largely lexical: the request names an operation and one candidate's documentation describes it, so even Laya reaches 94%. Banking77 is also matching, but across 77 fine-grained and partly overlapping intents, which separates CLEF-Flash from Jev. MMLU requires knowledge that is not present in the request, and routing requires judging how difficult and consequential a task is; on these, Laya and CLM fall to between 30% and 48% while the top two stay above 92%.

> *Matching a request to a described option is close to solved for the top models, and partly within reach of a small, fast model. Judging difficulty or recalling knowledge is where model capacity still decides the outcome.*

6.2 Threats to validity

Several factors limit how far these results generalise.

- **Diagnostic subsets.** Banking77 uses two messages per intent, ESCI ten sampled products per query, MMLU ten of 57 subjects, and BFCL scores only the function name. None of these are leaderboard scores.
- **Single run.** Each case was answered once per model, so run-to-run variation is not measured. Jev is a hosted model whose version can change; its Banking77 accuracy was 79.2% in an earlier two-model run against 80.5% here, while CLEF-Flash reproduced exactly.
- **Fixed routing order.** The routing question always lists *small*, *medium*, *large* in that order. Every provider saw the same order, so the comparison is fair, but position bias [22, 23] is not controlled on that suite.
- **Curated routing labels.** The routing gold labels are single curated judgments, and the alternative labels disagree on 5 of 90 prompts (Section 5.5).
- **Vendor configurations.** Laya ran with a larger token budget than its default; default settings would score lower on Banking77. CLM was evaluated as the zero-shot check-

point, while its model card attributes its published agentic results to fine-tuned heads. Reproducing the card's own example gave the same decision but a probability of 0.988 instead of the documented 0.939, so a small numerical difference in the encoder cannot be ruled out.

- **Recorded versions.** The public-suite rows predate per-row model identifiers, so Jev's version there (jev-1.13.0 in the routing and BFCL rows) is inferred from running in the same session. Model-weight revisions and two unpinned Python dependencies in the CLEF-Flash image were not recorded.
- **Latency.** Jev's figures include network time, and the local figures depend on this GPU, on request size, and on a host whose second GPU carried an unrelated workload.

6.3 Practical guidance

Within those limits, the results support a simple decision framework for choosing among these four models.

If the task has many fine-grained labels, as in intent triage $\rightarrow$ **CLEF-Flash**: it leads Jev by 13 points on Banking77, the only significant top-tier gap

If the decision needs knowledge or a difficulty judgement $\rightarrow$ **CLEF-Flash or Jev**: neither is significantly better on MMLU, ESCI, or routing

If tool selection must fit a tight latency budget $\rightarrow$ **Laya**: 94.0% at a 9 ms median, though 4.5 points below CLEF-Flash (Holm $p = 0.045$)

If probabilities will gate downstream actions $\rightarrow$ **check calibration per task**: CLEF-Flash is underconfident on routing and Jev overconfident on Banking77 errors

If only the zero-shot CLM checkpoint is available $\rightarrow$ **train task heads first**: it is at or near chance on Banking77, MMLU, and routing

6.4 Future work

Four extensions would sharpen these conclusions:

1. Shuffle the routing options per case and rerun, to measure position bias directly.
2. Repeat the hosted-model runs on several days to estimate version drift.
3. Evaluate full Banking77 and MMLU test sets, which would narrow the intervals in Table 7.
4. Score BFCL arguments as well as function names, and add a fine-tuned CLM for comparison.

7 Conclusion

Four decision models received identical requests through one protocol and one client, on five suites and 2,776 measured requests, with every per-case answer saved. CLEF-Flash and Jev form a clear top tier and are statistically tied on four suites; CLEF-Flash's 13-point lead on fine-grained intent triage is the one significant difference between them. Laya trades accuracy for speed and is competitive only on tool selection, and the zero-shot CLM checkpoint is near chance on three suites. Paired tests, interval estimates, and explicit baselines turned a table of accuracies into claims that can be defended, and the saved per-case rows let anyone recompute or extend every one of them.

A All pairwise McNemar tests

The two tables below list every McNemar test behind Section 5.2. "Left" and "right" follow alphabetical provider order, as in the saved result files.

Table 15: Exact McNemar tests for Banking77 and MMLU, Holm-adjusted within each suite

Suite	Left, right	Left only	Right only	p	p_{Holm}
Banking77	CLEF-Flash, CLM	142	0	3.6×10^{-43}	2.2×10^{-42}
Banking77	CLEF-Flash, Jev	21	1	1.1×10^{-5}	1.1×10^{-5}
Banking77	CLEF-Flash, Laya	75	1	2.0×10^{-21}	8.2×10^{-21}
Banking77	CLM, Jev	0	122	3.8×10^{-37}	1.9×10^{-36}
Banking77	CLM, Laya	2	70	1.1×10^{-18}	3.3×10^{-18}
Banking77	Jev, Laya	59	5	9.0×10^{-13}	1.8×10^{-12}
MMLU	CLEF-Flash, CLM	129	3	1.4×10^{-34}	7.0×10^{-34}
MMLU	CLEF-Flash, Jev	9	11	0.824	0.824
MMLU	CLEF-Flash, Laya	119	3	1.1×10^{-31}	3.4×10^{-31}
MMLU	CLM, Jev	0	128	5.9×10^{-39}	3.5×10^{-38}
MMLU	CLM, Laya	36	46	0.320	0.641
MMLU	Jev, Laya	120	2	2.8×10^{-33}	1.1×10^{-32}

Table 16: Exact McNemar tests for routing and BFCL, Holm-adjusted within each suite

Suite	Left, right	Left only	Right only	p	p_{Holm}
Routing	CLEF-Flash, CLM	55	0	5.6×10^{-17}	2.8×10^{-16}
Routing	CLEF-Flash, Jev	2	3	1.000	1.000
Routing	CLEF-Flash, Laya	47	3	3.7×10^{-11}	1.1×10^{-10}
Routing	CLM, Jev	0	56	2.8×10^{-17}	1.7×10^{-16}
Routing	CLM, Laya	14	25	0.108	0.216
Routing	Jev, Laya	47	2	4.4×10^{-12}	1.7×10^{-11}
BFCL	CLEF-Flash, CLM	68	2	4.2×10^{-18}	2.1×10^{-17}
BFCL	CLEF-Flash, Jev	0	3	0.250	0.250
BFCL	CLEF-Flash, Laya	11	2	0.0225	0.0449
BFCL	CLM, Jev	0	69	3.4×10^{-21}	2.0×10^{-20}
BFCL	CLM, Laya	7	64	1.3×10^{-12}	5.0×10^{-12}
BFCL	Jev, Laya	12	0	4.9×10^{-4}	1.5×10^{-3}

B MMLU accuracy by subject

Table 17 breaks MMLU accuracy down by subject. Each subject has 20 questions, so one question moves a score by five points.

Table 17: MMLU accuracy (%) by subject, 20 questions each

Subject	CLEF-Flash	Jev	Laya	CLM
Abstract algebra	90	95	20	40
Anatomy	75	85	20	30
Business ethics	95	80	45	20
College computer science	90	100	45	40
Conceptual physics	95	95	35	35
High school psychology	95	95	35	25
International law	100	100	35	25
Machine learning	95	95	25	10
Professional medicine	95	100	35	25
World religions	95	90	50	45

C Reproducing the analysis

Every statistic in this report is computed from the saved per-case rows, so the analysis can be reproduced offline without starting any model server (Code 3).

```bash
python3 -m pip install -r requirements.txt
python3 scripts/analyze_public_benchmarks.py
python3 scripts/build_results_notebook.py
jupyter nbconvert --to notebook --execute --inplace decision_model_benchmarks_results.
    ↪ ipynb
```

Code 3: Recomputing the statistics, notebook, figures, and tables from the saved results

Rerunning inference for one provider requires its server (or, for Jev, an API key) and replaces that provider's saved rows (Code 4).

```bash
python3 scripts/run_public_benchmarks.py --providers laya --force
python3 scripts/run_model_routing.py --providers laya --force
python3 scripts/run_tool_selection.py --providers laya --force
```

Code 4: Rerunning all three runners for one provider while its server is up

D References

[1] Cloudflare, "CLEF-Flash model card," https://huggingface.co/Cloudflare/clef-flash, 2026, accessed: 2026-10-11.

[2] TypeSafe, "Jev hosted SystemOne API," https://api.typesafe.ai/v1/systemone, 2026, model version reported by the API: jev-1.13.0. Accessed: 2026-10-11.

[3] ConvAI Innovations, "Laya model card and laya.serve package, version 0.3.21," https://huggingface.co/convaiinnovations/laya, 2026, accessed: 2026-10-11.

[4] Contrastive-LM, "CLM-v0.1-8B model card and contrastive-lm package, version 0.1.0," https://huggingface.co/Contrastive-LM/CLM-v0.1-8B, 2026, accessed: 2026-10-11.

[5] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[6] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024.

[7] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson, and I. Vulić, "Efficient intent detection

with dual sentence encoders," in *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*. Association for Computational Linguistics, 2020, pp. 38–45.

[8] C. K. Reddy, L. Màrquez, F. Valero, N. Rao, H. Zaragoza, S. Bandyopadhyay, A. Biswas, A. Xing, and K. Subbian, "Shopping queries dataset: A large-scale ESCI benchmark for improving product search," *arXiv preprint arXiv:2206.06588*, 2022.

[9] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021.

[10] Gorilla LLM, UC Berkeley, "Berkeley function calling leaderboard (BFCL) dataset, v3 multiple split," https://huggingface.co/datasets/gorilla-llm/Berkeley-Function-Calling-Leaderboard, 2024, accessed: 2026-10-09.

[11] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," *arXiv preprint arXiv:2305.15334*, 2023.

[12] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[13] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[14] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[15] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[16] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[17] C. Burges, T. Shaked, E. Renshaw, A. Lazier, M. Deeds, N. Hamilton, and G. Hullender, "Learning to rank using gradient descent," in *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 89–96.

[18] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[19] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[20] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using Bayesian binning," in *Proceedings of the 29th AAAI Conference on Artificial Intelligence*, 2015, pp. 2901–2907.

[21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.

[22] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, "Large language models are not robust multiple choice selectors," in *International Conference on Learning Representations (ICLR)*, 2024.

[23] P. Pezeshkpour and E. Hruschka, "Large language models sensitivity to the order of options in multiple-choice questions," in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.

[24] MTEB, "mteb/banking77: Banking77 test split in parquet form," https://huggingface.co/datasets/mteb/banking77, 2026, accessed: 2026-10-09.

[25] tasksource, "tasksource/esci: Shopping queries (ESCI) test split," https://huggingface.co/datasets/tasksource/esci, 2026, accessed: 2026-10-09.

[26] CAIS, "cais/mmlu: Massive multitask language understanding," https://huggingface.co/datasets/cais/mmlu, 2026, accessed: 2026-10-09.

[27] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[28] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Follow me for More AI Content

If you found these notes useful, connect with me on LinkedIn for more deep dives into Machine Learning, Artificial Intelligence, and Computer Vision.

Isham Rashik on LinkedIn

Scan the QR code or click the link above

